SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I Deva Dharshini(192472350) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Deva Dharshini

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;
   for(i = 0; i <= n; i++) {
      sum = sum + arr[i];
   }
   return sum;
}

int main() {
   int arr[5] = {1,2,3,4,5}
   int total;

   total = sumArray(arr);

   if(total == 15) {
      printf("Correct Sum\n")
   }

   int a = 10, b = 0;
   int c = a / b;
```

```
    int *p;
    *p = 20;

    return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error:
2. Specify the line number
3. Clearly describe the nature of the error

(b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

(c) Compilation Behavior (Evaluation Level)

1. Analyze which errors will be detected during compilation and which will only appear during execution.
2. Explain how early-stage errors (lexical/syntax) may prevent detection of later-stage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

(a) Error Identification

The given program contains different types of errors. The errors, their line numbers, and descriptions are given below.

Line No.	Error	Description
5	i <= n	The loop condition is incorrect because it accesses an element outside the array boundary.
13	Missing semicolon after array initialization	A semicolon is missing at the end of the array declaration.
16	sumArray(arr)	The function requires two arguments, but only one argument is passed.
19	Missing semicolon after printf()	A semicolon is missing at the end of the statement.
23	int c = a / b;	The divisor b is zero, which causes division by zero during execution.
26	*p = 20;	The pointer p is not initialized before dereferencing it.

(b) Classification of Errors

1. Lexical Errors

Lexical errors occur when the compiler encounters invalid characters or tokens. No lexical error is present in the given program.

2. Syntax Errors

Syntax errors occur when the rules of the C language are not followed.

- Missing semicolon after the array declaration.
- Missing semicolon after the printf() statement.

3. Semantic Errors

Semantic errors occur when statements are syntactically correct but their meaning or usage is incorrect.

- The function sumArray() requires two arguments, but only one argument is passed in the function call.

4. Logical Errors

Logical errors occur when the program logic is incorrect and may produce unexpected results.

- The loop condition i <= n should be i < n. Otherwise, the loop accesses an element beyond the array boundary.

5. Runtime Errors

Runtime errors occur while the program is executing.

- Division by zero occurs because b = 0.
- Dereferencing the uninitialized pointer p may cause a program crash or invalid memory access.

(c) Compilation Behavior

Some errors are detected during compilation, while others occur during program execution.

Error	**When it is detected**
Missing semicolons	During compilation
Incorrect number of function arguments	During compilation
Incorrect loop condition	May not be detected during compilation
Division by zero	May be detected by a compiler warning or occur during execution
Uninitialized pointer	May produce a compiler warning and cause an error during execution

Lexical analysis is the first phase of compilation, followed by syntax and semantic analysis. If a syntax error occurs, the compiler may not be able to analyze the affected statements correctly. Therefore, syntax errors should be corrected before checking the remaining errors.

Logical errors may not be detected by the compiler because the program can follow the language rules but still produce incorrect results.

(d) Corrected Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;

   for (i = 0; i < n; i++) {
      sum = sum + arr[i];
   }

   return sum;
}

int main() {
   int arr[5] = {1, 2, 3, 4, 5};
   int total;

   total = sumArray(arr, 5);

   if (total == 15) {
      printf("Correct Sum\n");
   }

   int a = 10, b = 2;
   int c = a / b;

   int value = 0;
   int *p = &value;
   *p = 20;

   printf("Total = %d\n", total);
   printf("Division result = %d\n", c);
   printf("Pointer value = %d\n", value);

   return 0;
}
```

Output:

Correct Sum

Total = 15

Division result = 5

Pointer value = 20

(e) Impact Analysis

1. Improper pointer usage

An uninitialized pointer does not point to a known valid memory location. Accessing memory through it may cause a program crash or memory corruption.

2. Division by zero

Dividing an integer by zero causes undefined behavior in C. The program may terminate abnormally or produce unpredictable behavior.

3. Incorrect loop condition

The condition i <= n allows the loop to access an array element beyond its valid range. This may result in incorrect output or a program crash.

4. Missing semicolon

A missing semicolon causes a syntax error and prevents the program from compiling successfully until the error is corrected.

5. Incorrect function call

Passing fewer arguments than required causes a compilation error when the compiler checks the function call. The program cannot be compiled successfully until the call is corrected.

(f) Advanced Compiler Perspective

A compiler can be improved in the following ways:

1. Better error messages
    - Display the exact line number and location of the error.
    - Explain the error in simple language.
    - Suggest possible corrections for common mistakes.
2. Early detection of runtime errors
    - Detect possible division-by-zero errors.
    - Identify uninitialized pointers and variables.
    - Check array boundaries to prevent invalid memory access.
3. Improved debugging tools
    - Static analysis: Examines the program without executing it and identifies possible errors.
    - Debugging symbols: Help programmers identify source-code lines and variable values during debugging.
    - Runtime checking: Detects memory errors and invalid operations while the program is running.

These improvements help programmers find errors quickly, reduce debugging time, and develop more reliable programs.

Q2.The following C program contains multiple lexical errors introduced intentionally. These errors occur due to invalid tokens, incorrect identifiers, malformed constants, and illegal symbols, which are typically detected during the lexical analysis phase of a compiler. Carefully examine the program and answer the questions given below.

```
#include <stdio.h>
int main() {
    int 1num = 100;
    float rate@ = 25.5;
    char ch = 'abc';
    int total = 50$20;
    int num#1 = 30;
    int %count = 40;
    printf("Welcome to Compiler Design);
    int x = 09;
    int y = 0xZ12;
    int z = 12.3.4;

    char str1[] = "Hello;
    char str2[] = "World";

    int _valid = 10;
    int invalid-id = 20;

    float value = 5..6;
    int data = 10@5;

    char c = 'xy';
    int 3value = 60;

    printf("Value: %d\n", total);

    return 0;
}
```

Tasks

(a) Error Identification

i) Identify all lexical errors in the program.

ii) List the line number and the invalid token.

(b) Error Classification

i) For each error, explain why it is considered a lexical error.

ii) Specify which rule of token formation is violated:

iii) Identifier rules

iv) Constant formation

v) String/character literal rules

vi) Operator/symbol rules

(c) Token Correction

i)Rewrite each invalid token into a valid token.

ii)Example: 2value → value2

(d) Compiler Behavior Analysis

i) Explain how a lexical analyzer (scanner) processes such errors.

ii)Discuss whether the compiler:

iii)Stops immediately

iv)Continues scanning using error recovery

(e) Advanced Thinking

i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.

ii) Suggest how error messages can be improved for beginners.

(a) Error Identification

The following table shows the lexical errors present in the given program.

Line No.	**Invalid token**	**Error**
4	1num	Identifier starts with a digit.
5	rate@	Contains an illegal symbol @.
6	'abc'	Character constant contains more than one character.
7	50$20	Contains an illegal symbol $.

8	num#1	Contains an illegal symbol #.
9	%count	Identifier starts with an illegal symbol %.
11	"Welcome to Compiler Design)	Missing closing double quotation mark.
13	09	Invalid octal integer constant because octal digits range from 0 to 7.
14	0xZ12	Invalid hexadecimal constant because Z is not a hexadecimal digit.
15	12.3.4	Invalid floating-point constant with two decimal points.
17	"Hello;	Missing closing double quotation mark.
20	invalid-id	Contains an illegal hyphen in an identifier.
22	5..6	Invalid floating-point constant with two decimal points.
23	10@5	Contains an illegal symbol @.
25	'xy'	Character constant contains more than one character.
26	3value	Identifier starts with a digit.

(b) Error Classification

1. Identifier Rules

In C, an identifier must begin with a letter or an underscore. It can contain letters, digits, and underscores, but it cannot contain spaces or special symbols such as @, #, %, or -.

Examples:

- 1num — starts with a digit.
- rate@ — contains @.
- num#1 — contains #.
- %count — starts with %.
- invalid-id — contains a hyphen.
- 3value — starts with a digit.

2. Constant Formation

Constants must follow the rules of the C language.

- 50$20 — contains an invalid symbol $.
- 09 — invalid octal constant because octal digits range from 0 to 7.
- 0xZ12 — invalid hexadecimal constant because Z is not a hexadecimal digit.
- 12.3.4 — contains two decimal points.
- 5..6 — contains two decimal points.
- 10@5 — contains an invalid symbol @.

3. String and Character Literal Rules

String literals must begin and end with double quotation marks. Character constants are normally written using single quotation marks around one character.

- "Welcome to Compiler Design) — missing the closing double quotation mark.
- "Hello; — missing the closing double quotation mark.
- 'abc' — contains multiple characters in a character constant.
- 'xy' — contains multiple characters in a character constant.

4. Operator and Symbol Rules

C identifiers and numeric constants cannot contain arbitrary special symbols.

- @ and $ are not valid parts of C identifiers or numeric constants.
- # is used in preprocessing directives and cannot be used inside an ordinary identifier.
- % is an operator in C, not a valid first character of an identifier.
- - is an operator and cannot be used inside an identifier.

(c) Token Correction

The following table shows possible corrections for the invalid tokens.

Invalid token	Corrected token
1num	num1
rate@	rate
'abc'	"abc"
50$20	5020
num#1	num1
%count	count
"Welcome to Compiler Design)	"Welcome to Compiler Design"
09	9
0xZ12	0x12
12.3.4	12.34
"Hello;	"Hello"
invalid-id	invalid_id
5..6	5.6
10@5	105
'xy'	"xy"
3value	value3

(d) Compiler Behavior Analysis

1. How a Lexical Analyzer Processes Errors

The lexical analyzer, also called a scanner, is the first phase of a compiler. It reads the source program character by character and groups the characters into tokens such as keywords, identifiers, operators, constants, and string literals.

When it encounters an invalid token, it reports a lexical error and may continue scanning the remaining program.

2. Does the Compiler Stop Immediately?

The compiler does not always stop immediately after finding a lexical error. Its behavior depends on the compiler implementation and the type of error.

- Stops immediately: Some compilers stop when an error prevents further analysis.
- Continues scanning: Many compilers report an error, skip invalid characters or tokens, and continue scanning to identify additional errors.

- Error recovery: The scanner attempts to recover and resume analysis from a suitable point in the source program.

For example, if the scanner encounters 1num, it can report an invalid identifier and continue processing the next line.

(e) Advanced Thinking

1. DFA Idea to Detect Identifiers Starting with Digits

A Deterministic Finite Automaton (DFA) can be designed to identify identifiers that begin with digits.

Rules:

1. If the first character is a letter or underscore, continue checking the identifier.
2. If the first character is a digit, enter an invalid-identifier state.
3. If the next characters contain letters, digits, or underscores, report an invalid identifier beginning with a digit.
4. Stop checking when a character that cannot belong to the token is encountered.

Example:

- 2value — invalid identifier.
- value2 — valid identifier.
- _value — valid identifier.
- num_2 — valid identifier.

2. Improving Error Messages for Beginners

A compiler can provide better error messages by:

- Displaying the exact line number and column number.
- Explaining why the token is invalid.
- Showing the correct identifier or constant format.
- Suggesting a corrected token.
- Highlighting the invalid character.

Example:

Instead of displaying:

Lexical error

The compiler can display:

Line 4: Invalid identifier '1num'. An identifier cannot start with a digit. Try 'num1'.

These improvements help beginners understand lexical errors and correct their programs easily.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

A. Lexical Errors
B. Syntax Errors
C. Semantic Errors
D. Intermediate Code Generation Issues (logic/precedence)
E. Code Optimization Issues (redundancy/inefficiency)
F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b
}

int main() {
   int #value = 20;
   float num = 15.5
   int result;

   result = divide(10);

   if(result = 5) {
      printf("Result is 5\n")
   }

   int x = 10;
   int y = 0;
   int z = x / y;
```

```
    int arr[4] = {1,2,3,4};
    printf("%d\n", arr[4]);

    int *ptr;
    *ptr = 25;

    int total;
    total = x + y * result + 0;

    while(x < 20) {
        printf("%d\n", x);
    }

    return 0;
}
```

Do the following Tasks

(a) Identify Errors (Analysis Level)

I. Identify **all errors** in the program.

II. Mention **line number + error description**.

(b) Classify by Compiler Phase

Classify each error into:

I. Lexical (invalid symbols like @num)

II. Syntax (missing semicolon, brackets)

III. Semantic (function argument mismatch)

IV. Intermediate Code Issues (wrong expression handling/precedence)

V. Optimization Issues (redundant + 0)

VI. Runtime Errors (division by zero, invalid pointer access)

(c) Compilation vs Execution (Evaluation)

I. Which errors are detected at **compile time**?

II. Which errors appear only during **execution**?

III. Explain why some errors prevent further compilation.

(d) Correct the Program (Application)

I. Rewrite the **fully corrected program**.

II. Ensure:

   A. Proper syntax

   B. Correct semantics

   C. Safe execution

(e) Deep Analysis (Evaluation)

I. Explain how **incorrect pointer usage** affects runtime.

II. Analyze how **array bounds violation** impacts memory.

III. Discuss how **operator precedence errors** affect intermediate code.

(f) Advanced Task (Create Level)

I. Suggest **2 improvements** in compiler design to:

   A. Detect runtime errors earlier

   B. Improve debugging messages

II. Propose how **optimization phase** can remove redundant expressions.

(a) Identify Errors

The given C program contains errors in syntax, lexical analysis, semantic analysis, intermediate code generation, optimization, and runtime execution.

Line No.	**Error**	**Description**
4	return a / b	Missing semicolon at the end of the return statement.
9	int #value = 20;	The identifier contains an invalid symbol #.
10	float num = 15.5	Missing semicolon at the end of the declaration.
14	result = divide(10);	The function requires two arguments, but only one is passed.
16	if(result = 5)	Assignment operator = is used instead of comparison operator ==.

17	printf("Result is 5\n")	Missing semicolon at the end of the statement.
23	int z = x / y;	Division by zero because y = 0.
26	arr[4]	The array has valid indices from 0 to 3. Index 4 is out of bounds.
29	*ptr = 25;	The pointer is uninitialized before dereferencing.
32	total = x + y * result + 0;	Adding zero is redundant and can be optimized.
34–36	while(x < 20)	The value of x is never updated, so the loop continues indefinitely if entered.

(b) Classify by Compiler Phase

1. Lexical Errors

- Line 9: #value contains an invalid symbol within an identifier. In C, # cannot be used as part of an ordinary identifier.

2. Syntax Errors

- Line 4: Missing semicolon after return a / b.
- Line 10: Missing semicolon after float num = 15.5.
- Line 17: Missing semicolon after printf().

3. Semantic Errors

- Line 14: The function divide() requires two arguments, but only one argument is supplied.
- Line 16: The assignment result = 5 inside the condition changes the value of result instead of comparing it with 5. This is usually a logic error rather than a syntax error.

4. Intermediate Code Generation Issues

- Line 16: The assignment inside the condition may produce incorrect conditional behavior because it assigns a value rather than comparing values.
- Line 32: The expression x + y * result + 0 contains a redundant addition. The multiplication is evaluated before addition according to operator precedence.

The compiler normally handles operator precedence correctly. The expression itself is valid C; the problem is the possible intention of the programmer and the unnecessary operation.

5. Code Optimization Issues

- Line 32: Adding zero does not change the value of the expression. An optimizing compiler can remove + 0.

- Line 10: The variable num is initialized but never used. An optimizing compiler may remove unnecessary code related to it.

6. Runtime Errors

- Line 23: Division by zero.
- Line 26: Accessing an array element outside its valid range.
- Line 29: Dereferencing an uninitialized pointer.
- Lines 34–36: The loop becomes infinite if x remains less than 20.

(c) Compilation vs Execution

1. Errors Detected at Compile Time

- Invalid identifier #value.
- Missing semicolons.
- Incorrect number of function arguments.

These errors are generally detected during lexical, syntax, or semantic analysis. They prevent the program from compiling successfully until corrected.

2. Errors That Occur During Execution

- Division by zero.
- Accessing an array element outside its valid range.
- Dereferencing an uninitialized pointer.
- Infinite looping caused by the unchanged value of x.

Some compilers may warn about these problems, but they are not all guaranteed to be detected during compilation.

3. Why Some Errors Prevent Further Compilation

The compiler processes the program in several stages. Lexical analysis identifies tokens, syntax analysis checks grammar, and semantic analysis checks meaning and types.
If an early-stage error occurs, later stages may not be able to process the affected code correctly. The compiler may stop or attempt error recovery to report additional errors.

(d) Corrected Program

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b;
}

int main() {
```

```
int value = 20;
float num = 15.5;
int result;

result = divide(10, 2);

if (result == 5) {
   printf("Result is 5\n");
}

int x = 10;
int y = 2;
int z = x / y;

int arr[4] = {1, 2, 3, 4};
printf("%d\n", arr[3]);

int value2 = 0;
int *ptr = &value2;
*ptr = 25;

int total;
total = x + y * result;

while (x < 20) {
   printf("%d\n", x);
   x++;
}

printf("Division result = %d\n", z);
printf("Pointer value = %d\n", *ptr);
printf("Total = %d\n", total);

return 0;
```

}

Output

Result is 5

4

10

11

12

13

14

15

16

17

18

19

Division result = 5

Pointer value = 25

Total = 20

(e) Deep Analysis

1. Effect of Incorrect Pointer Usage

A pointer stores the address of a variable. In the given program, ptr is declared but not initialized before the statement *ptr = 25.

Therefore, it may contain an invalid address. Accessing memory through this pointer can cause a program crash, memory corruption, or unpredictable behavior.

The problem is corrected by assigning the address of a valid variable to the pointer before dereferencing it.

2. Effect of Array Bounds Violation

The array arr[4] contains four elements, whose valid indices are 0, 1, 2, and 3.

Accessing arr[4] reads memory outside the array. This causes undefined behavior and may produce incorrect output, corrupt data, or crash the program.

3. Effect of Operator Precedence on Intermediate Code

In the expression:

total = x + y * result + 0;

Multiplication has higher precedence than addition. Therefore, the compiler evaluates the multiplication first.

The expression is interpreted as:

total = x + (y * result) + 0;

The intermediate representation can then be simplified to:

total = x + (y * result);

Removing the redundant addition reduces unnecessary operations while preserving the result.

(f) Advanced Task

1. Improvements to Detect Runtime Errors Earlier

Improvement 1: Static analysis

The compiler can analyze the program without executing it and warn about possible division by zero, uninitialized pointers, and array boundary violations.

Improvement 2: Runtime checking

The compiler can insert checks during compilation to detect invalid memory access, division by zero, and other unsafe operations when the program runs.

2. Improvements to Debugging Messages

- Display the exact line and column number of an error.
- Explain the cause of the error in simple language.
- Suggest possible corrections.
- Identify the variable or expression responsible for the problem.

For example, instead of displaying only an error message, the compiler can report that an array index is outside the valid range and show the permitted indices.

3. Removing Redundant Expressions During Optimization

The optimization phase can simplify expressions using techniques such as constant folding and algebraic simplification.

For example:

total = x + y * result + 0;

can be simplified to:

total = x + y * result;

Since adding zero does not change the result, the unnecessary operation can be removed. This reduces the number of operations and improves code efficiency.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand